

Button Function

Experiment Objective

In this experiment, you will learn how to read the user button state on the MicroROS-V2 control board. Through this example, you will master the configuration of GPIO input mode, the use of internal pull-up resistors, and the implementation principle of button debouncing.

Experiment Principle

1. Keywords

Keyword	Description
GPIO input mode	Configures the GPIO as input to read the high/low level signal of an external device
Internal pull-up resistor	A pull-up resistor integrated inside the ESP32 that keeps the pin at a high level when floating
Active low	Pressing the button pulls the GPIO low to GND; detecting a low level indicates the button is pressed
Software debounce	Eliminates the multiple level transitions caused by mechanical button bounce using a software algorithm (state machine/delay)
State machine	A programming model that determines the next state based on the current state and input, used for button debouncing

2. Technical Architecture

The program initializes GPIO42 as input mode and enables the internal pull-up, then enters the main loop and checks the button state every 10 ms. The button driver uses a non-blocking state machine to implement software debouncing, ensuring each press triggers the event only once. Key files are located in `~/esp/base_samples/key/`, including `main/main.c` (main program entry), `components/key/key.h` (pin definitions and function declarations) and `components/key/key.c` (button driver implementation).

Experiment Steps

1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	 Required
USB Type-C data cable	 Required

2. Prerequisites

1. ESP32-IDF development environment set up
2. MicroROS-V2 control board connected to the PC via USB
3. Confirm the user button (labeled KEY) on the control board is functioning properly

3. Compile and Flash

```
cd ~/esp/base_samples/key
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

4. How to Stop

Press `Ctrl+]` in the terminal to exit the serial monitor.

Experiment Result

After compiling and flashing successfully, the serial monitor will output:

```
hello yahboom
I (xxx) KEY_MAIN: Nice to meet you!
```

When the user button on the control board is pressed, the output is:

```
I (xxx) KEY_MAIN: KEY 1 PRESS
free heap size = 2450528
```

Each time the button is pressed, the terminal outputs the message only once; after releasing and pressing again, it can be triggered repeatedly.

Code Explanation

Source code paths:

- Firmware source code: `~/esp/base_samples/key/main/main.c`
- Button driver: `~/esp/base_samples/key/components/key/key.h`, `key.c`

1. Main Program Logic

`main/main.c` is the program entry, and the flow is as follows:

1. Print the startup message `"hello yahboom"` and a welcome log
2. Call `key_Init()` to initialize the button GPIO
3. Enter the main loop and check the button state every 10 ms:
 - Read the button state via `key1_Read_State()`
 - When a button press is detected, print `"KEY 1 PRESS"` and the current free heap size

2. Key Components

`components/key/key.h` — pin definitions and function declarations:

Parameter definition file: `~/esp/base_samples/key/components/key/key.h`

```
#define KEY_GPIO_USER1      42    // User button GPIO
#define KEY_STATE_PRESS     1     // Pressed state
#define KEY_STATE_RELEASE   0     // Released state
```

`components/key/key.c` — button driver implementation, with the following core functions:

Function	Description
<code>Key_Init()</code>	Initialize GPIO42 as input mode and enable the internal pull-up resistor
<code>Key1_Read_State()</code>	Non-blocking read of the button state with software debounce; returns <code>KEY_STATE_PRESS</code> once per button press

Button hardware principle:

The button circuit is **active low**. When the button is not pressed, thanks to the internal pull-up resistor (`pull_up_en = 1`), the level read on GPIO42 is high; when the button is pressed, GPIO42 is pulled to GND and the level read is low.

```
// Check whether the button is pressed (internal function)
static uint8_t Key1_is_Pressed(void)
{
    uint8_t key_state = KEY_STATE_RELEASE;
    if (!gpio_get_level(KEY_GPIO_USER1)) // Reading a low level means pressed
    {
        key_state = KEY_STATE_PRESS;
    }
    return key_state;
}
```

Button initialization:

`Key_Init()` uses the `gpio_config_t` structure to configure the GPIO:

```
gpio_config_t io_conf = {};
io_conf.intr_type = GPIO_INTR_DISABLE;           // Disable interrupt
io_conf.mode = GPIO_MODE_INPUT;                  // Input mode
io_conf.pin_bit_mask = (1ULL<<42);              // GPIO42
io_conf.pull_down_en = 0;                        // Disable pull-down
io_conf.pull_up_en = 1;                          // Enable pull-up
gpio_config(&io_conf);
```

Software debounce principle:

`Key1_Read_State()` implements debouncing with a non-blocking state machine. The core logic is as follows:

- Internally maintains a 3-state counter (0, 1, 2)
- Each time a button press (low level) is detected, the counter increments, up to a maximum of 3
- When the count reaches 2, `KEY_STATE_PRESS` is returned once
- If a button release (high level) is detected, the counter is cleared immediately
- This approach ensures the press event is triggered only when the button is stably pressed, effectively filtering out mechanical bounce

Common Problems

Problem	Cause	Solution
Button does not respond	The button GPIO pin is not 42, or the pull-up is not enabled	Check the <code>KEY_GPIO_USER1</code> macro in <code>key.h</code> and the pull-up configuration in <code>Key_Init()</code>
One press triggers multiple times	Mechanical bounce causes unstable levels	Check whether the debounce logic is working properly and ensure the <code>vTaskDelay(10ms)</code> loop period is correct
No serial output	Abnormal USB serial connection	Check the USB cable connection and confirm <code>idf.py monitor</code> starts normally